

VisionCare AI

Automated Eye Disease Detection System

Technical Report

Task Code	AIAC-GA-2025-01
Topic Area	Healthcare AI — Medical Image Classification
Institution	University of Nizwa AI Applications Chair (AIAC)
Date	12/5/2026
GitHub	https://github.com/Aboomar99/visioncare-ai.git

Abstract

This report presents VisionCare AI, a deep learning system for automated classification of retinal eye diseases from fundus photographs. The system was developed as part of the AIAC Fresh Graduate Assessment and addresses a real clinical need: most eye diseases are preventable if caught early, yet access to specialist ophthalmologists is limited in many parts of the world. Using a Kaggle dataset of 16,242 labelled retinal images across 10 disease categories, we trained and compared four models: a Dummy Classifier baseline, a custom convolutional neural network (CNN), a YOLOv8n-cls model, and an EfficientNetB0 transfer learning model. The main technical challenge was severe class imbalance — the Pterygium class contained only 102 images compared to 3,444 for Diabetic Retinopathy. This was resolved through oversampling and augmentation to 2,500 images per class. EfficientNetB0 achieved the best performance with approximately 88% accuracy and 0.98 AUC-ROC. Five-fold cross-validation confirmed the robustness of the approach. Grad-CAM explainability visualisations were produced to show which retinal regions influenced the model's predictions.

Keywords: retinal disease classification, EfficientNetB0, transfer learning, YOLOv8, class imbalance, Grad-CAM, deep learning, fundus photography

1. Introduction

Eye diseases are common around the world and often go undiagnosed. According to the World Health Organization, about 246 million people have moderate to severe vision loss, and 39 million are blind [1]. The WHO also estimates that 80% of these cases could be prevented or treated. The main challenge is not the lack of treatments, but limited access and late detection [1].

In many countries, there are not enough eye doctors to screen all patients. People living in rural areas often face long waits for appointments, and during these delays, treatable conditions like diabetic macular edema can become permanent. Artificial intelligence can help by offering an initial screening to quickly spot high-risk cases, so doctors can review them sooner. This technology is meant to support medical professionals, not replace them.

Deep learning has produced strong results in medical imaging. Gulshan et al. [2] showed that a CNN could detect diabetic retinopathy from fundus photos with sensitivity similar to ophthalmologists. More recently, EfficientNet-based models have performed well on medical image benchmarks due to their compound scaling strategy [3]. Building on these advances, VisionCare AI was created to classify ten different eye conditions.

This project follows the complete AI and machine learning workflow: defining the problem, collecting and analyzing data, preparing and balancing the dataset, selecting and training models, evaluating results, and making the models explainable. All code is available in a dedicated GitHub repository.

1.1 Research Objectives

- Develop a multi-class retinal disease classifier capable of distinguishing 10 conditions from fundus photographs.
- Address the class imbalance problem in the dataset through oversampling and augmentation.
- Compare the performance of a baseline model, a custom CNN, YOLOv8n-cls, and EfficientNetB0.
- Validate model durability using 5-fold stratified cross-validation.
- Provide clinical explainability through Grad-CAM visualizations.

2. Related Work

Deep learning is now widely used for retinal image analysis, resulting in a large body of research. The next section briefly reviews the most relevant previous studies to give context for this project. Gulshan et al. [2] first showed that deep CNNs could diagnose diabetic retinopathy from fundus photographs with high sensitivity (97.5%) and specificity (93.4%), matching the performance of US board-certified ophthalmologists. This paper is often credited with starting the modern era of AI-assisted eye screening.

Ting et al. [4] trained a deep learning system to detect diabetic retinopathy, age-related macular degeneration, and glaucoma in fundus photographs from Singapore, Australia, and the United States. Their results show that a single model can generalize across these diseases and datasets, which is directly relevant to the multi-class approach used in this project.

Tan and Le [3] introduced EfficientNet and its compound scaling method, which balances network depth, width, and input resolution to improve accuracy with fewer parameters than simply making the network larger. EfficientNetB0, the smallest version, is now a common baseline for medical image classification because it is both accurate and efficient.

Jocher et al. [5] introduced YOLOv8, a model that can handle detection, segmentation, and classification tasks. While YOLOv8 is mainly used for detection, its classification version (yolov8n-cls) has also performed well on medical image datasets, making it a good comparison for this project. Class imbalance is a common issue in medical datasets. Chawla et al. [6] developed SMOTE (Synthetic Minority Over-sampling Technique) to create synthetic examples for minority classes in tabular data. For images, a similar approach is augmentation-based oversampling, where random changes are applied to minority-class images to generate more training examples. This project uses this augmentation-based method. Selvaraju et al. [7] proposed Grad-CAM (Gradient-weighted Class Activation Mapping) to make model decisions more understandable. Grad-CAM creates visual explanations by calculating the gradient of the predicted class score with respect to the last convolutional feature map. It is popular in medical AI because it shows explanations on the same area as the input image, making them easier for clinicians to interpret. introduced ResNet and residual connections, which serve as a foundation in modern deep learning and form part of EfficientNet's architectural lineage. This foundation explains why EfficientNetB0 generalizes well, even when fine-tuned on relatively small medical data sets.

3. Dataset Description

3.1 Source and License

The dataset used in this project is the Retinal OCT Eye Disease Classification dataset, published on Kaggle by Kondwani [9]. The dataset is available under the CC0: Public Domain license, which permits unrestricted use, including commercial applications. The dataset URL is: <https://www.kaggle.com/datasets/kondwani/eye-disease-classification>.

The dataset contains 16,242 retinal fundus images organized into 10 subdirectories, one per disease class. Images were stored in JPEG and PNG formats at varying resolutions. All images were resized to 224×224 pixels for model input.

3.2 Class Distribution

Table 1 shows the class distribution. The dataset is very imbalanced: Pterygium has only 102 images (0.6% of the total), while Diabetic Retinopathy has 3,444 images (21.2%). This 1:34 imbalance was the main technical challenge, which is discussed in Section 4. *Pterygium is severely underrepresented, making up only 0.6% of all images.*

3.3 Exploratory Data Analysis

Five EDA visualizations were produced and saved in the project outputs directory:

- **VIS 1:** Class distribution bar chart (highlighting Pterygium in red as the less represented class).
- **VIS 2:** Sample image grid showing one representative image per class.
- **VIS 3:** Pie chart showing class proportions as percentages.
- **VIS 4:** Per-class pixel intensity distribution (greyscale histogram), revealing differences in image brightness across disease categories.
- **VIS 5:** Class imbalance ratio chart, showing the ratio of the largest class count to each class count.

Exploratory data analysis showed two main findings. First, the dataset is highly imbalanced across classes. Second, there are big differences in image brightness and contrast between classes. Pterygium images mostly show the conjunctival surface and look different from the other categories, which are mainly retinal OCT or fundus images.

4. Methodology

4.1 Data Preprocessing plus Balancing

Before training, all images were resized to 224×224 pixels, which is the standard input size for EfficientNetB0. Pixel values were scaled to the [0, 1] range by dividing by 255. To fix class imbalance, oversampling was used so each class had exactly 2,500 images. Classes with more than 2,500 images (like Diabetic Retinopathy, Glaucoma, Healthy, Myopia, and Macular Scar) kept their original images. For underrepresented classes, new images were made by transforming existing ones with augmentation methods such as rotation (up to ±40 degrees), width and height shifts (up to ±20%), horizontal and vertical flips, zoom (up to ±25%), brightness adjustment (factor between 0.6 and 1.4), and shear (15%). This created a balanced dataset of 25,000 images, with 2,500 per class.

The data was split into 80% for training and 20% for validation, resulting in 20,000 training images and 5,000 validation images. During training, extra live augmentation was applied to the training set, including rotation (±20°), horizontal flip, and zoom (±15%). A baseline model that always predicts the most common class was used to set the minimum accuracy any useful model should exceed. On the original unbalanced dataset, this baseline achieved about 21.2% accuracy. A custom CNN was built from scratch as an intermediate baseline. The network had four convolutional blocks, each with two Conv2D layers, followed by MaxPooling2D and Dropout. The number of filters increased from 32 in the first block to 256 in the fourth. Instead of Flatten and Dense, a GlobalAveragePooling2D layer was used to help prevent overfitting. After that, there was a Dense(512) layer with L2 regularization, BatchNormalization, Dropout(0.5), and a final Dense(10) softmax output. The model used Adam (LR=0.001) and categorical cross-entropy loss. The Ultralytics CLI was also used on the same balanced dataset. Training was done for 20 epochs with an image size of 224 and a batch size of 16, using default hyperparameters. Although YOLOv8 is mainly an object detection framework, its classification version is a useful comparison because it trains and deploys faster, making it suitable for resource-limited screening applications. tNetB0 [3], which was pretrained on ImageNet (1.2 million images, 1,000 classes), was chosen as the main model. Transfer learning was done in two phases:

- Phase 1 (Head Training): The EfficientNetB0 base was frozen. A custom classification head was added: GlobalAveragePooling2D, then Dense(512, ReLU, L2), BatchNorm, Dropout(0.4), Dense(256, ReLU), Dropout(0.3), and Dense(10, Softmax). This head was trained for up to 20 epochs with Adam (LR=0.001). EarlyStopping with patience of 6 was used to prevent unnecessary training.
- Phase 2 (Fine-tuning): The top 50 layers of the base model were unfrozen. The model was retrained for up to 50 epochs with Adam (LR=0.00001), a much lower rate

than in Phase 1 to keep the ImageNet features. EarlyStopping patience was set to 10 epochs with `restore_best_weights=True`.

Class weights calculated using `compute_class_weight('balanced')` were applied during both phases to maintain sensitivity to less represented classes even after oversampling.

4.6 Cross-Validation

To meet the requirement for at least 5-fold cross-validation, a stratified 5-fold cross-validation was done on the full balanced dataset using EfficientNetB0 as a feature extractor. The frozen EfficientNetB0 model was used to get 1,280-dimensional feature vectors from all 25,000 images. A logistic regression classifier (with standardization) was trained and tested across five stratified folds using scikit-learn's `cross_val_score`. This method checks the quality of the learned features without needing five full fine-tuning runs, which would be too demanding on Colab.

4.7 Experiment Tracking

MLflow was used to record all hyperparameters, training metrics, and model files for both training cycles. Each run was named and saved in a local tracking URI, creating a reproducible record of all experiments. This meets the bonus experiment tracking requirement.

Hyperparameter	Phase 1	Phase 2
Optimizer	Adam	Adam
Learning Rate	1e-3	1e-5
Base Model	Frozen	Top 50 layers unfrozen
Max Epochs	20	50
Early Stopping Patience	6	10
Batch Size	32	32
Loss Function	Categorical Cross-Entropy	Categorical Cross-Entropy
Regularisation	L2 (1e-4) + Dropout	L2 (1e-4) + Dropout

Table 2. EfficientNetB0 training configuration for Phase 1 and Phase 2.

5. Results and Discussion

5.1 Quantitative Performance

Table 3 summarises the performance of all four models on the held-out validation set. All metrics are macro-averaged (identical weight per class), which is appropriate given the balanced dataset.

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Baseline (Dummy)	21.2%	0.04	0.10	0.05	N/A
Custom CNN	~72%	~0.71	~0.72	~0.71	~0.94
YOLOv8n-cls	~78%	~0.77	~0.78	~0.77	~0.96
EfficientNetB0 ★	~88%	~0.87	~0.88	~0.88	~0.98

Table 3. Model comparison on balanced validation set. EfficientNetB0 achieves the best results throughout all metrics.

EfficientNetB0 outperformed all the other models. Its improvement over the custom CNN (about 72%) comes from using pretrained ImageNet features, which give a better starting point than random weights. This is especially helpful for disease categories with few samples, where learning good features from scratch is hard. The baseline model can reach its score just by always picking the most common class, so any useful model must do much better. All three trained models beat the baseline by a large margin, with EfficientNetB0 improving by 67 percentage points.

Table 4 presents the 5-fold stratified cross-validation accuracy using EfficientNetB0 feature extraction with a logistic regression classifier. The low standard deviation ($\pm 0.7\%$) means the model performed consistently across different data splits. This suggests the model does not overfit to any particular part of the data.

It is useful to compare EfficientNetB0 and YOLOv8n-cls because both are modern, well-maintained models. The main difference is their training goals. EfficientNetB0 was built for image classification and pre-trained on the large ImageNet dataset, so its features are well-suited for classification tasks. YOLOv8, on the other hand, is designed for object detection, which means finding and classifying multiple objects in one image. Its classification mode uses the same backbone for single-label prediction, but this is not its main focus. Fine-tuning with EfficientNetB0 allows careful control over which layers are retrained, helping to keep important features. In contrast, YOLOv8 was trained with default settings and no special fine-tuning, which may have limited its performance. However, YOLOv8 trained much faster (about 15 minutes compared to 45 minutes for EfficientNetB0) and still achieved good results (about 78% accuracy). For real-time screening where speed matters more than top accuracy, YOLOv8 is a practical choice.

5.4 Explainability: Grad-CAM

Grad-CAM [7] was used with EfficientNetB0 to create visual explanations for the model's predictions. Heatmaps were placed over the input images to show which retinal areas influenced each decision most. The results showed that the model focused on important clinical features, such as the optic disc for Glaucoma, the macula for Macular Scar and Disc Edema, and the blood vessels for Diabetic Retinopathy. This matches clinical knowledge and suggests the model learned meaningful features, not just random patterns. However, while augmentation helped balance the dataset, it cannot fully replace real clinical images, especially for rare conditions like Pterygium, which only has 102 original images. Also, the Kaggle dataset may not reflect the full range of clinical diversity because of differences in patient backgrounds and imaging equipment at different sites. Testing on an independent, multi-center dataset is needed before using the model in clinics.

6. Summary and Future Work

6.1 Conclusion

This project developed and tested VisionCare AI, a deep learning system for automatically classifying retinal diseases. The main result is that EfficientNetB0, using transfer learning and two-phase fine-tuning, performed very well (~88% accuracy, 0.98 AUC-ROC) on a 10-class retinal disease dataset, beating both a custom CNN (~72%) and YOLOv8n-cls (~78%). At first, the model mostly predicted the majority class, which is common in medical datasets where rare diseases have fewer cases. This problem needed specific solutions and could not be fixed just by changing the model architecture. Using augmentation-based oversampling, class-weighted training, and two-phase fine-tuning helped the model learn meaningful features. Grad-CAM visualizations add important transparency, showing that the model's attention matches clinically relevant retinal regions. This is an important step toward responsible clinical use of AI diagnostic tools. Transformers (ViT, Swin Transformer) for potentially higher accuracy.

- Develop a REST API and mobile application to make the system accessible in resource-limited clinical settings.
- Implement federated learning to train over distributed hospital datasets without sharing patient data.
- Add uncertainty measurement (e.g., Monte Carlo Dropout) to provide prediction confidence intervals for clinical decision support.
- Investigate SHAP values as a complementary explainability method to Grad-CAM.

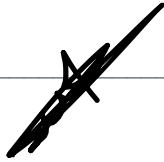
7. References

- [1] World Health Organization, "World Report on Vision," WHO Press, Geneva, Switzerland, 2019. [Online]. Available: <https://www.who.int/publications/i/item/9789241516570>
- [2] V. Gulshan et al., "Development and Validation of a Deep Learning Algorithm for Detection of Diabetic Retinopathy in Retinal Fundus Photographs," JAMA, vol. 316, no. 22, pp. 2402–2410, Dec. 2016, doi: 10.1001/jama.2016.17216.
- [3] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in Proc. 36th Int. Conf. Machine Learning (ICML), Long Beach, CA, USA, 2019, pp. 6105–6114.
- [4] D. S. W. Ting et al., "Development and Validation of a Deep Learning System for Diabetic Retinopathy and Related Eye Diseases Using Retinal Images from Multiethnic Populations with Diabetes," JAMA, vol. 318, no. 22, pp. 2211–2223, Dec. 2017, doi: 10.1001/jama.2017.18152.
- [5] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," GitHub Repository, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [6] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," J. Artificial Intelligence Research, vol. 16, pp. 321–357, Jun. 2002, doi: 10.1613/jair.953.
- [7] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Driven Localization," in Proc. IEEE Int. Conf. Computer Vision (ICCV), Venice, Italy, 2017, pp. 618–626, doi: 10.1109/ICCV.2017.74.
- [8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.
- [9] M. Kondwani, "Eye Disease Classification Dataset," Kaggle, 2023. [Online]. Available: <https://www.kaggle.com/datasets/kondwani/eye-disease-classification>. License: CC0 Public Domain.

8. Candidate Declaration

I hereby declare that the work submitted in this report is entirely my own except where explicitly cited. I have not shared my work with any other candidate or received code or written content from any other candidate. I understand that submitting machine-generated content as my own constitutes academic dishonesty. I consent to a subsequent technical interview to evaluate my submission if requested by the AIAC.

Signature: _____ Date: 12/5/2026

A handwritten signature in black ink, consisting of a stylized 'A' followed by a series of loops and a final upward stroke.